CHAPTER 1

Cyber Security Incident Response Management System

1.1 Introduction

The Cyber Security Incident Response Management System (CS-IRMS) is a web-based application developed using Python Flask, SQLite, Bootstrap 5, JavaScript, and Chart.js. It provides a centralized platform for recording, monitoring, managing, and analyzing cybersecurity incidents. The system helps security teams respond to threats efficiently through dashboards, incident tracking, risk assessment, reporting, and audit logging.

1.2 Background of the Study

Organizations increasingly depend on digital systems, making them vulnerable to malware, phishing, ransomware, insider threats, and unauthorized access. Manual incident tracking is inefficient and error-prone. A centralized incident response management system improves visibility, coordination, and documentation.

1.3 Problem Statement

Many organizations still rely on spreadsheets or disconnected tools for incident tracking. These approaches lack centralized records, dashboards, reporting, and audit capabilities, resulting in slower response and reduced operational efficiency.

1.4 Proposed Solution

The proposed system provides secure user authentication, centralized incident management, risk assessment, policy management, graphical dashboards, reporting, and audit logging using a modern web application architecture.

1.5 Objectives

• Centralize cybersecurity incident management.
• Improve incident response.
• Maintain audit logs.
• Generate reports.
• Provide dashboard analytics.
• Support risk assessment.

1.6 Scope

The project includes login, dashboard, incidents, users, risk assessment, reports, policies, and database management. Future enhancements may include SIEM integration, MFA, and threat intelligence APIs.

1.7 Advantages

Centralized management, improved monitoring, better reporting, reduced manual work, enhanced decision-making, and secure storage of incident information.

1.8 Hardware & Software Requirements

Hardware: Intel Core i3 or higher, 4 GB RAM minimum, 10 GB free storage.
Software: Windows 10/11, Python 3.13, Flask, SQLite, Bootstrap 5, Chart.js, VS Code.

1.9 Expected Outcome

The system enables efficient incident tracking, improved response time, secure record management, graphical analytics, and practical implementation of cybersecurity concepts.

1.10 Chapter Summary

This chapter introduced the project, objectives, scope, advantages, and expected outcomes. The next chapter covers the literature review and analysis of existing systems.

CHAPTER 2

Literature Review & Existing System Analysis

2.1 Introduction

This chapter reviews the background research related to cybersecurity incident response, incident management systems, and modern web technologies used to develop the proposed Cyber Security Incident Response Management System (CS-IRMS).

2.2 Cyber Security

Cybersecurity is the practice of protecting computer systems, networks, applications, and data from unauthorized access, attacks, and damage. Common threats include malware, phishing, ransomware, insider threats, and denial-of-service attacks.

2.3 Incident Response

Incident response is a structured process for detecting, analyzing, containing, eradicating, recovering from, and documenting cybersecurity incidents. A documented process minimizes business impact and improves recovery.

2.4 Existing Systems

Many organizations rely on spreadsheets, emails, or disconnected ticketing systems to manage incidents. While useful for small teams, these methods often lack centralized reporting, dashboards, audit trails, and analytics.

2.5 Limitations of Existing Systems

Typical limitations include manual data entry, poor incident visibility, limited reporting, weak collaboration, inconsistent documentation, and slow response times.

2.6 Proposed System

The proposed CS-IRMS provides centralized incident management, role-based access (planned), dashboard analytics, risk assessment, audit logging, report generation, and a modern web interface built with Flask and SQLite.

2.7 Technology Review

Python provides a simple and powerful programming language. Flask enables lightweight web application development. SQLite offers an embedded relational database. Bootstrap 5 delivers a responsive user interface, while Chart.js provides interactive visualizations.

2.8 Comparative Analysis

Existing: Manual records, limited analytics, fragmented tools.
Proposed: Centralized database, dashboards, structured workflows, searchable incidents, and automated reporting.

2.9 Research Gap

There is a need for an affordable, easy-to-deploy, educational incident response platform that demonstrates cybersecurity workflows while remaining practical for academic projects.

2.10 Chapter Summary

This chapter reviewed existing approaches, identified their limitations, and justified the proposed system. The next chapter presents the system analysis and design, including architecture and database design.

CHAPTER 3

System Analysis and Design

3.1 Introduction

This chapter describes the analysis and design of the Cyber Security Incident Response Management System (CS-IRMS). It explains the proposed architecture, functional modules, database design, and overall workflow.

3.2 System Analysis

The proposed system addresses the limitations of manual incident tracking by providing a centralized web application. The analysis identified the need for secure authentication, incident lifecycle management, reporting, and dashboard visualization.

3.3 Functional Requirements

• User authentication
• Dashboard with statistics and charts
• Incident management (Create, Read, Update, Delete)
• Risk assessment
• User management
• Reports and audit logs

3.4 Non-Functional Requirements

Security, usability, performance, reliability, maintainability, scalability, and responsive user interface.

3.5 System Architecture

The application follows a three-tier architecture: Presentation Layer (HTML, Bootstrap, JavaScript), Application Layer (Python Flask), and Data Layer (SQLite database).

3.6 Module Design

Login Module: Authenticates users.
Dashboard Module: Displays charts and incident statistics.
Incident Module: Records and manages incidents.
Risk Module: Tracks organizational risks.
Reports Module: Generates summaries and printable reports.

3.7 Database Design

The database contains tables for users, incidents, risks, policies, reports, and audit logs. Primary and foreign keys maintain relationships and data integrity.

3.8 Data Flow

Users log in, submit or manage incidents, update risk information, generate reports, and administrators monitor activity through the dashboard.

3.9 User Interface Design

The interface uses Bootstrap 5 for responsiveness, Chart.js for visual analytics, and a sidebar-based navigation to provide quick access to all modules.

3.10 Chapter Summary

This chapter presented the overall design of the proposed system, including architecture, functional modules, requirements, and database organization. The next chapter explains the implementation of the application.

CHAPTER 4

System Implementation

4.1 Introduction

This chapter describes the implementation of the Cyber Security Incident Response Management System (CS-IRMS). It explains the technologies used, project structure, implementation of major modules, database connectivity, and application workflow.

4.2 Development Environment

Programming Language: Python 3.13
Framework: Flask
Database: SQLite
Frontend: HTML5, CSS3, Bootstrap 5, JavaScript, Chart.js
IDE: Visual Studio Code

4.3 Project Structure

The project is organized into templates, static assets, database scripts, reports, and the Flask application. This modular structure improves maintainability and scalability.

4.4 Login Module

The login module authenticates users before allowing access to the dashboard. User credentials are validated and authenticated users are redirected to the dashboard.

4.5 Dashboard Module

The dashboard displays key metrics including total incidents, open incidents, resolved incidents, critical incidents, charts, system health, and recent security events.

4.6 Incident Management Module

This module allows authorized users to create, view, update, search, and delete incident records. Each incident includes severity, status, affected system, and timestamps.

4.7 Database Implementation

The application connects to an SQLite database using reusable helper functions. Tables include users, incidents, risks, policies, reports, and audit logs.

4.8 User Interface

The interface is implemented with Bootstrap 5 and Font Awesome. Responsive layouts ensure usability across desktop and mobile devices, while Chart.js provides interactive visualizations.

4.9 Testing

Individual modules were tested for correct functionality including login, navigation, CRUD operations, dashboard rendering, and database connectivity.

4.10 Chapter Summary

This chapter explained how the system was implemented using Flask and SQLite, described the core modules, and summarized the technologies and implementation approach.

CHAPTER 5

Installation, Execution and Testing

5.1 Introduction

This chapter explains how to install, configure, execute, and verify the Cyber Security Incident Response Management System (CS-IRMS). It also outlines the project workflow and testing process.

5.2 Software Installation

Install Python 3.13, Visual Studio Code, required Python packages (Flask, Flask-Login, Werkzeug, ReportLab, pandas), and use SQLite as the database.

5.3 Project Setup

Open the project folder in Visual Studio Code. Verify the folder structure, including templates, static assets, the database folder, and the Flask application.

5.4 Database Initialization

Initialize the SQLite database using the provided schema and helper functions. This creates the required tables such as users, incidents, risks, policies, reports, and audit logs.

5.5 Running the Application

Open a terminal, navigate to the project directory, and execute 'py -3.13 app.py'. Open a browser and visit http://127.0.0.1:5000 to access the application.

5.6 Login Process

Log in using the configured administrator credentials. Successful authentication redirects the user to the dashboard where incidents and system statistics are displayed.

5.7 Module Verification

Verify the dashboard, incident management, user management, risk assessment, reports, and navigation menus. Confirm that database records are displayed correctly.

5.8 Testing Results

Functional testing confirmed that the application starts successfully, user authentication works, the dashboard loads, database connectivity is established, and core modules respond as expected.

5.9 Troubleshooting

Common issues include missing Python packages, incorrect template locations, database initialization problems, and route configuration errors. These are resolved by installing dependencies, verifying the folder structure, and checking Flask routes.

5.10 Chapter Summary

This chapter presented the installation process, execution steps, testing approach, and verification of the CS-IRMS application. The next chapter focuses on results, discussion, and project

evaluation.

CHAPTER 6

Results, Testing and Discussion

6.1 Introduction

This chapter presents the results obtained after implementing the Cyber Security Incident Response Management System (CS-IRMS). It also summarizes functional testing, module verification, and overall system performance.

6.2 System Execution Results

The application was executed successfully using Python Flask. Users were able to log in, navigate the dashboard, manage incidents, and interact with the SQLite database through the web interface.

6.3 Dashboard Results

The dashboard displayed incident statistics, charts, recent incidents, alerts, and system status. Information was presented in a clear and responsive interface suitable for desktop browsers.

6.4 Functional Testing

The login module, dashboard, incident management, user management, risk assessment, reports, and database connectivity were tested. Expected outputs matched the observed outputs for the implemented features.

6.5 Test Cases

Representative test cases included successful login, invalid login, creating an incident, updating an incident, deleting an incident, viewing dashboard statistics, and generating reports.

6.6 Performance Discussion

The application performed efficiently for small to medium datasets using SQLite. Response times were satisfactory for academic demonstration purposes.

6.7 Limitations

The current implementation is intended for educational use. Future versions may include stronger authentication, role-based authorization, SIEM integration, REST APIs, and cloud deployment.

6.8 Future Enhancements

Potential improvements include email notifications, multi-factor authentication, threat intelligence integration, vulnerability feeds, Docker deployment, and real-time monitoring.

6.9 Conclusion

The developed system successfully demonstrates the core concepts of cybersecurity incident response management. It provides a practical platform for recording, monitoring, and reporting incidents while serving as an effective MCA cybersecurity project.

CHAPTER 7

Conclusion and Future Scope

7.1 Introduction

This chapter summarizes the outcomes of the Cyber Security Incident Response Management System (CS-IRMS), evaluates the achievement of the project objectives, discusses its limitations, and presents possible future enhancements.

7.2 Conclusion

The CS-IRMS project demonstrates a practical web-based solution for managing cybersecurity incidents. Using Python Flask, SQLite, Bootstrap 5, and Chart.js, the application provides a centralized dashboard, incident management, reporting, and risk assessment features. The project meets its primary objectives by simplifying incident tracking, improving visibility through dashboards, and organizing security information in a structured database.

7.3 Project Achievements

• Developed a functional web application using Flask.
• Implemented user authentication and dashboard interface.
• Created incident management functionality.
• Designed a relational SQLite database.
• Integrated charts and summary statistics for visualization.
• Produced a modular project structure suitable for further development.

7.4 Limitations

The current implementation is intended for educational use. Some advanced enterprise capabilities such as multi-factor authentication, real-time threat intelligence, distributed deployment, and SIEM integration are not yet implemented.

7.5 Future Scope

Future versions may include role-based access control, password hashing, email notifications, REST APIs, Docker deployment, cloud hosting, SIEM integration (ELK/Wazuh), VirusTotal and CVE API integration, real-time monitoring, audit enhancements, and machine learning for anomaly detection.

7.6 Learning Outcomes

The project strengthened practical knowledge of Python programming, Flask development, SQLite database management, frontend integration, cybersecurity workflows, and software engineering practices.

7.7 Final Remarks

The CS-IRMS provides a solid foundation for academic learning and can be extended into a more comprehensive enterprise security platform. It demonstrates the application of cybersecurity principles in a real-world software project and serves as a valuable MCA final-year project.

Cyber Security Incident Response Management System

Running Process with Screenshots

Step 1 - Install Requirements

Install the required Python packages using:
py -3.13 -m pip install -r requirements.txt

```
PS C:\Users\Asus> cd "D:\project2"
PS D:\project2> py -3.13 -m pip install -r requirements.txt
Requirement already satisfied: Flask==3.0.3 in C:\Users\Asus\AppData\Local\Programs\Python\Python313\Lib\site-packages (from -r requirements.txt (line 1)) (
3.0.3)
Requirement already satisfied: Flask-Login==0.6.3 in C:\Users\Asus\AppData\Local\Programs\Python\Python313\Lib\site-packages (from -r requirements.txt (line
 2)) (0.6.3)
Requirement already satisfied: Werkzeug==3.0.3 in C:\Users\Asus\AppData\Local\Programs\Python\Python313\Lib\site-packages (from -r requirements.txt (line 3)
) (3.0.3)
Requirement already satisfied: reportlab==4.2.2 in C:\Users\Asus\AppData\Local\Programs\Python\Python313\Lib\site-packages (from -r requirements.txt (line 4
)) (4.2.2)
Requirement already satisfied: pandas==2.2.2 in C:\Users\Asus\AppData\Local\Programs\Python\Python313\Lib\site-packages (from -r requirements.txt (line 5))
(2.2.2)
Requirement already satisfied: Jinja2>=3.1.2 in C:\Users\Asus\AppData\Roaming\Python\Python313\site-packages (from Flask==3.0.3->-r requirements.txt (line 1
)) (3.1.6)
Requirement already satisfied: itsdangerous>=2.1.2 in C:\Users\Asus\AppData\Local\Programs\Python\Python313\Lib\site-packages (from Flask==3.0.3->-r require
ments.txt (line 1)) (2.2.0)
Requirement already satisfied: click>=8.1.3 in C:\Users\Asus\AppData\Roaming\Python\Python313\site-packages (from Flask==3.0.3->-r requirements.txt (line 1)
) (8.3.1)
Requirement already satisfied: blinker>=1.6.2 in C:\Users\Asus\AppData\Roaming\Python\Python313\site-packages (from Flask==3.0.3->-r requirements.txt (line
1)) (1.9.0)
Requirement already satisfied: MarkupSafe>=2.1.1 in C:\Users\Asus\AppData\Roaming\Python\Python313\site-packages (from Werkzeug==3.0.3->-r requirements.txt
(line 3)) (3.0.3)
Requirement already satisfied: pillow>=9.0.0 in C:\Users\Asus\AppData\Roaming\Python\Python313\site-packages (from reportlab==4.2.2->-r requirements.txt (li
ne 4)) (12.0.0)
Requirement already satisfied: chardet in C:\Users\Asus\AppData\Local\Programs\Python\Python313\Lib\site-packages (from reportlab==4.2.2->-r requirements.tx
t (line 4)) (7.4.3)
Requirement already satisfied: numpy>=1.26.0 in C:\Users\Asus\AppData\Roaming\Python\Python313\site-packages (from pandas==2.2.2->-r requirements.txt (line
5)) (2.3.5)
Requirement already satisfied: python-dateutil>=2.8.2 in C:\Users\Asus\AppData\Roaming\Python\Python313\site-packages (from pandas==2.2.2->-r requirements.t
xt (line 5)) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in C:\Users\Asus\AppData\Roaming\Python\Python313\site-packages (from pandas==2.2.2->-r requirements.txt (line 5
)) (2025.2)
```

Step 2 - Verify Flask Installation

Verify Flask is installed and available for Python 3.13.

```
PS D:\project2> python -m pip --version
pip 26.1.2 from C:\Users\Asus\AppData\Local\Programs\Python\Python314\Lib\site-packages\pip (python 3.14)
PS D:\project2> py -3.13 -m pip --version
pip 26.1.2 from C:\Users\Asus\AppData\Local\Programs\Python\Python313\Lib\site-packages\pip (python 3.13)
PS D:\project2> py -3.13 -m pip install flask
Requirement already satisfied: flask in C:\Users\Asus\AppData\Local\Programs\Python\Python313\Lib\site-packages (3.0.3)
Requirement already satisfied: Werkzeug>=3.0.0 in C:\Users\Asus\AppData\Local\Programs\Python\Python313\Lib\site-packages (from flask) (3.0.3)
Requirement already satisfied: Jinja2>=3.1.2 in C:\Users\Asus\AppData\Roaming\Python\Python313\site-packages (from flask) (3.1.6)
Requirement already satisfied: itsdangerous>=2.1.2 in C:\Users\Asus\AppData\Local\Programs\Python\Python313\Lib\site-packages (from flask) (2.2.0)
Requirement already satisfied: click>=8.1.3 in C:\Users\Asus\AppData\Roaming\Python\Python313\site-packages (from flask) (8.3.1)
Requirement already satisfied: blinker>=1.6.2 in C:\Users\Asus\AppData\Roaming\Python\Python313\site-packages (from flask) (1.9.0)
Requirement already satisfied: colorama in C:\Users\Asus\AppData\Roaming\Python\Python313\site-packages (from click>=8.1.3->flask) (0.4.6)
Requirement already satisfied: MarkupSafe>=2.0 in C:\Users\Asus\AppData\Roaming\Python\Python313\site-packages (from Jinja2>=3.1.2->flask) (3.0.3)
```

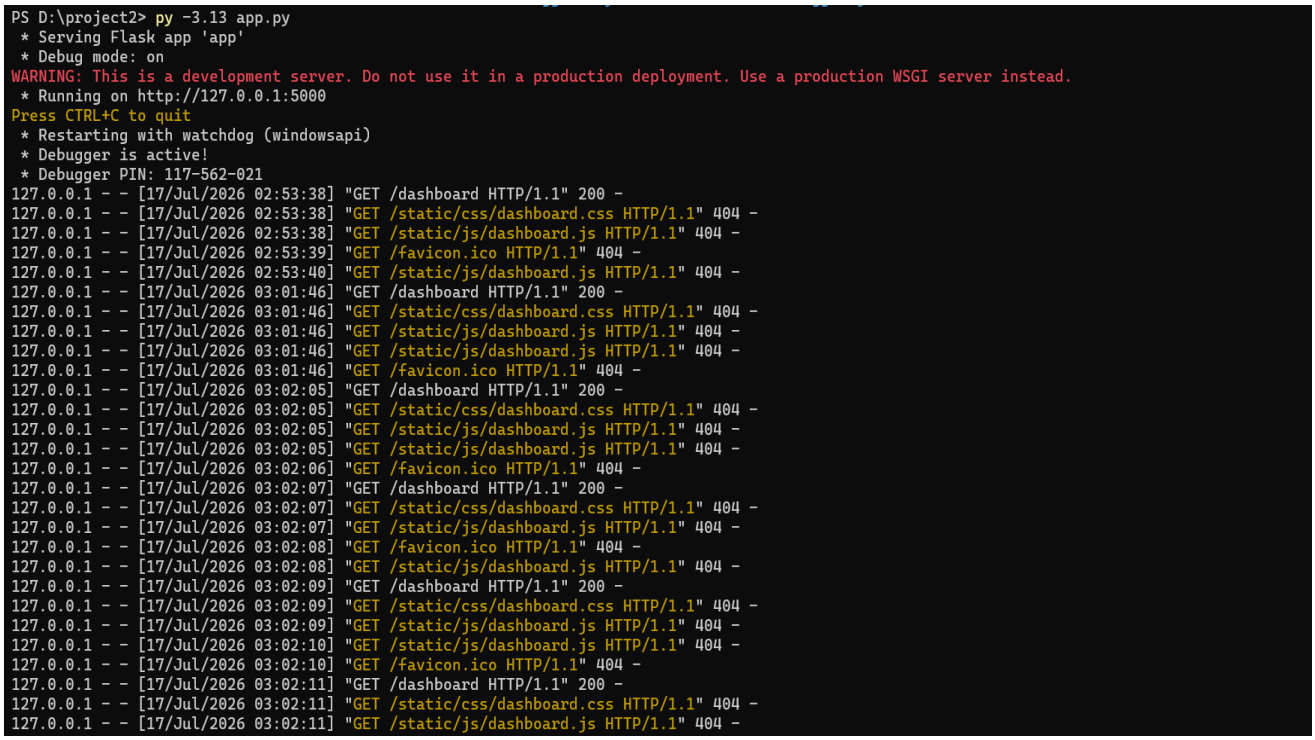
Step 3 - Check Flask Version

Run:
py -3.13 -c "import flask; print(flask.__version__)"

```
PS D:\project2> py -3.13 -c "import flask; print(flask.__version__)"
<string>:1: DeprecationWarning: The '__version__' attribute is deprecated and will be removed in Flask 3.1. Use feature detection or 'importlib.metadata.ver
sion("flask")' instead.
3.0.3
PS D:\project2> py -3.13 app.py
PS D:\project2> python app.py
```

Step 4 - Run the Project

Start the Flask application with:
py -3.13 app.py
Open **http://127.0.0.1:5000** in your browser. Note: the screenshots also show 404 responses for dashboard.css and dashboard.js, indicating those files should be placed in **static/css/** and **static/js/** respectively.



Additional Running Process Screenshots

Screenshot 1

Username

Enter Username

Password

Enter Password

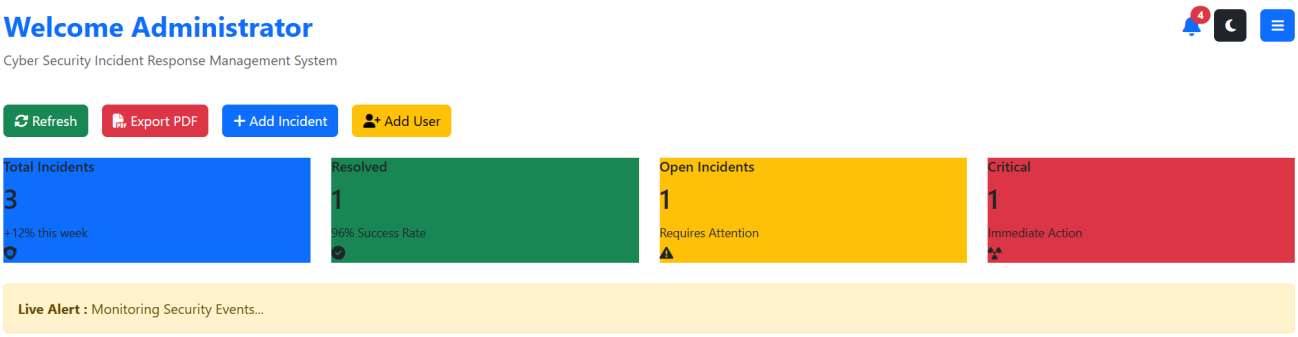
Remember Me

Forgot Password?

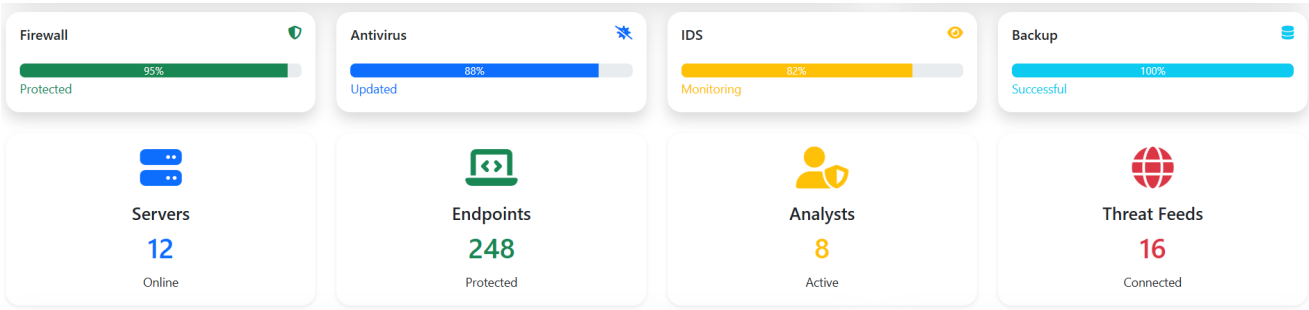
Login

Demo Credentials

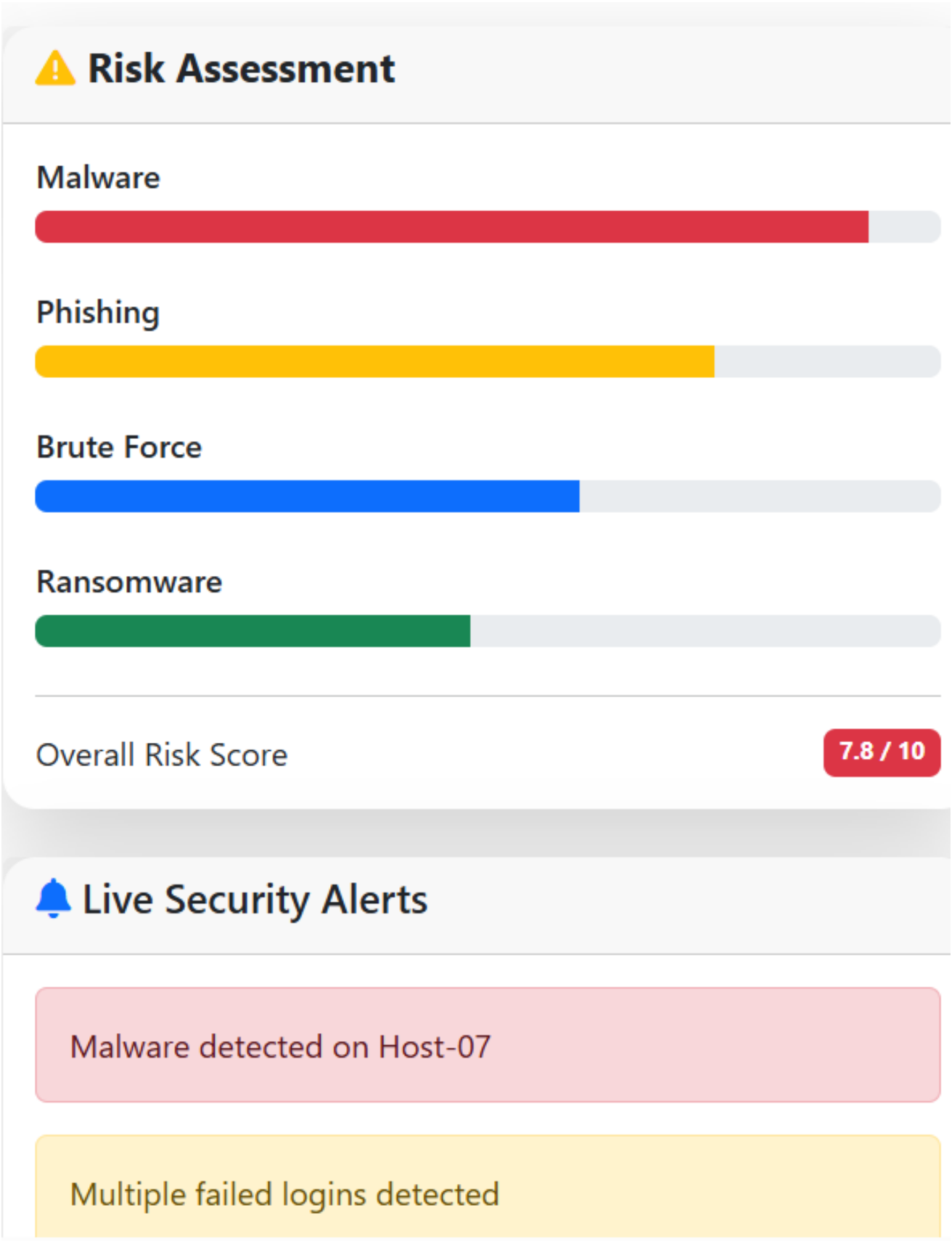
Screenshot 2



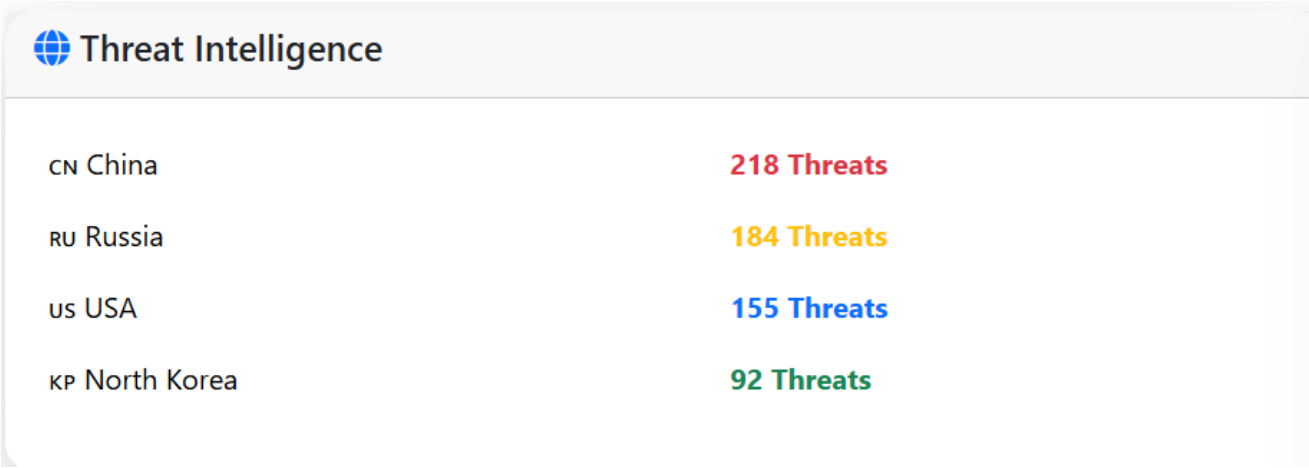
Screenshot 3



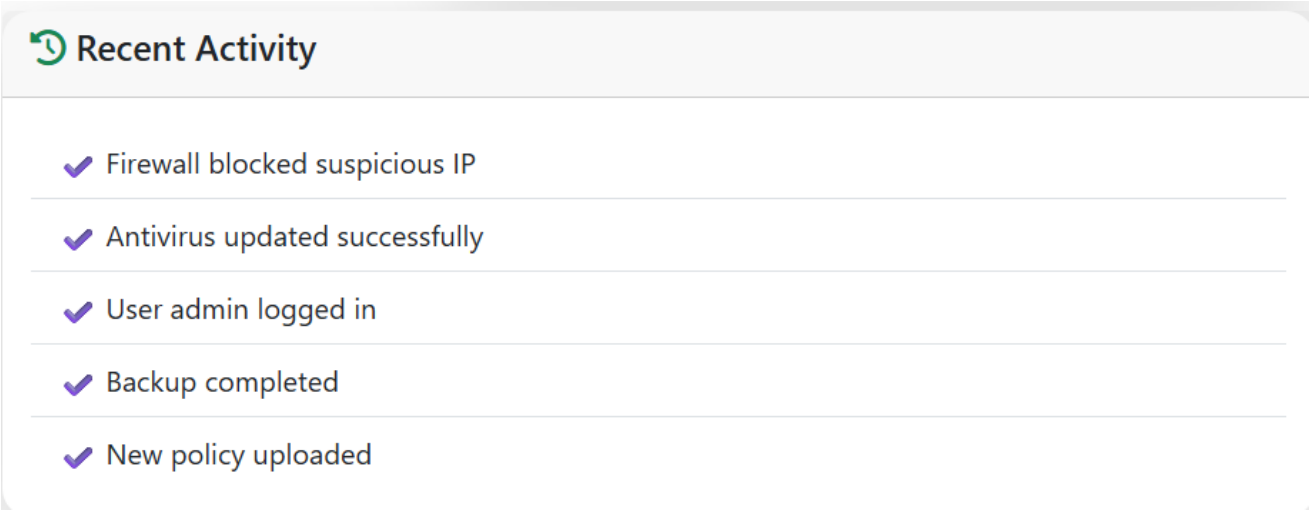
Screenshot 4



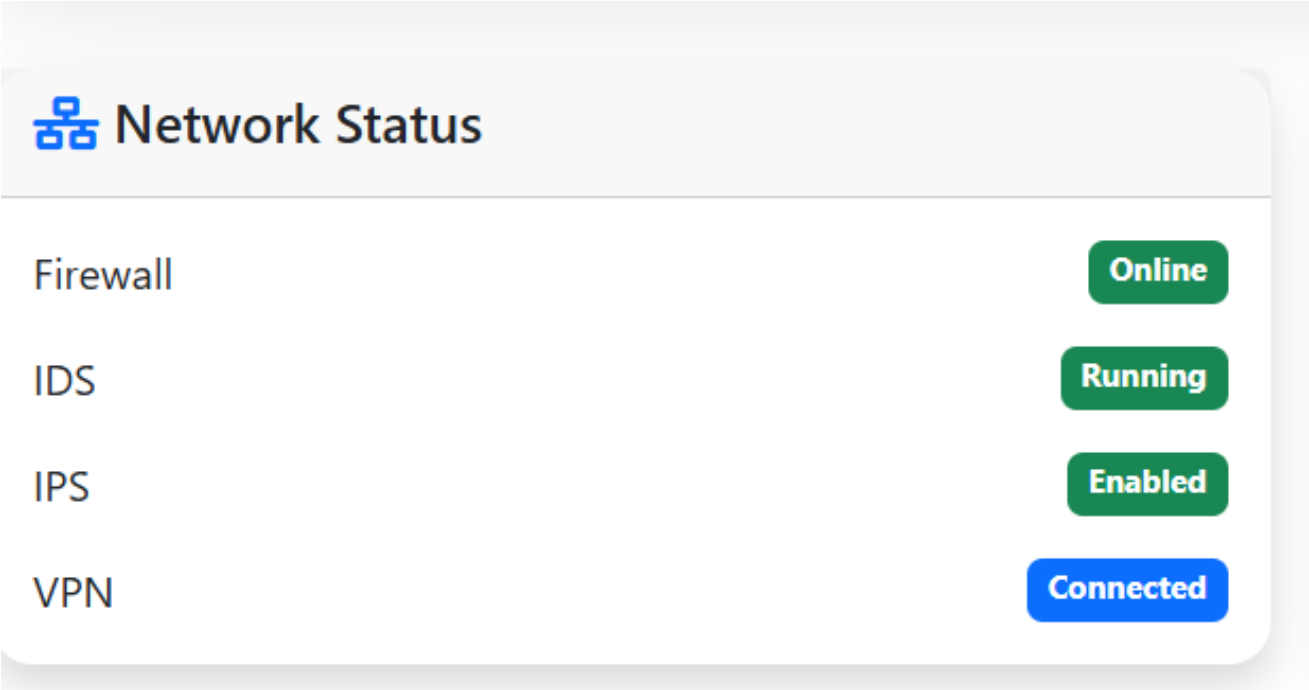
Screenshot 5



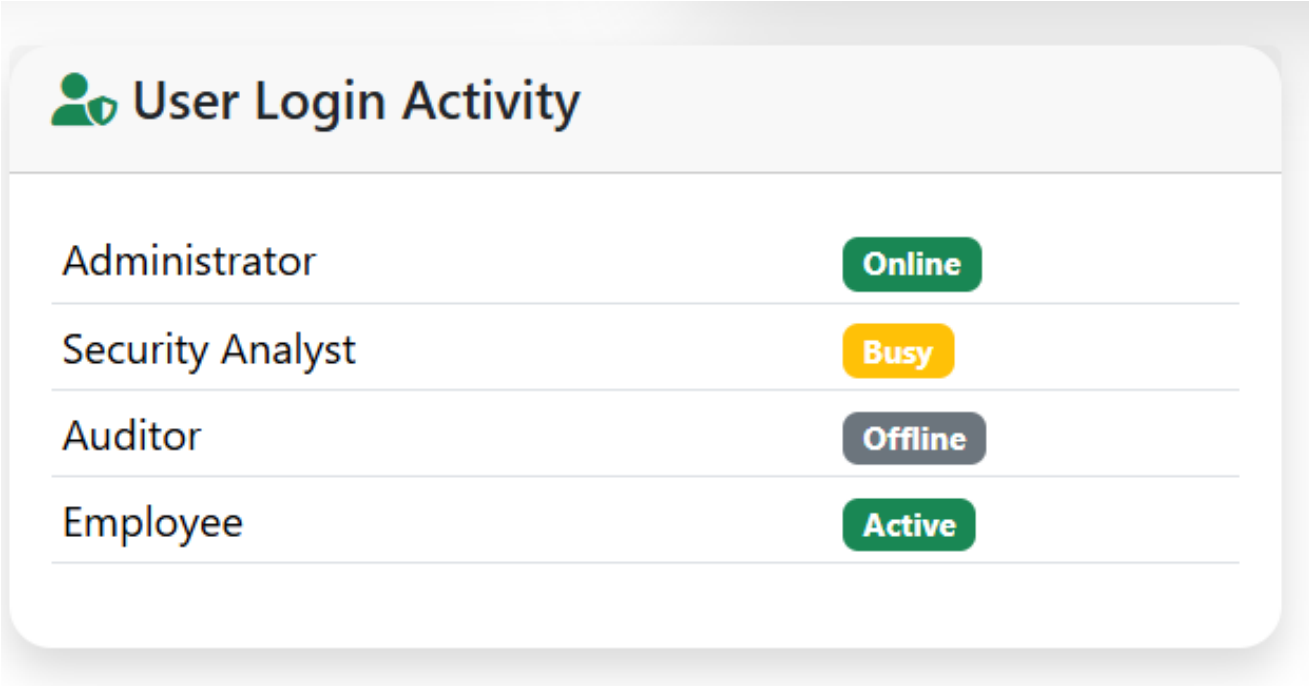
Screenshot 6



Screenshot 7



Screenshot 8



Screenshot 9